

[ReportNow] Design Report on Software Maintainability

Version 1.0

By: Ang Wei Ming, Chi Seo Hyeon, Choo Kean Yee, Dylan Quek Zhi En, Lim Jun Howe, Madelyn Cruz Tan, Tan Yong Huat

Contents

1. Design Strategies	3
1.1. Planning	3
1.2. Development	3
1.3. Correction	3
1.3.1. Preventive Maintenance	3
1.3.2. Corrective Maintenance	3
1.4. Enhancement	4
1.4.1. Adaptive Maintenance	4
1.4.2. Perfective Maintenance	4
1.5. Best Practices	4
2. System Architecture	5
2.1. Model Layer	5
2.2. View Layer	5
2.3. Controller Layer	5

1. Design Strategies

1.1. Planning

From the beginning of the planning phase, the team discussed potential improvements and upgrades to be implemented after the product's deployment. This is regarding the maintainability and scalability of the project. With the vision of a long-term deployment and usage, we agreed that the two mentioned factors were crucial aspects to plan for.

As the functionality of our product heavily relies on user interaction, the team planned to incorporate the Model-View-Controller model in our program architecture. This would allow convenient modularisation during the development phase, which would eventually result in enhanced maintainability and scalability in the long run.

1.2. Development

In the development phase, tests against the functional requirements would be conducted with a small, private pool of users. This is in consideration of our development environment and in respect to our deployment procedures, in which it is difficult to collect a large number of testers to simulate macro-scale traffic.

As for usability, since ReportNow aims to provide a user-friendly environment for users of varying technological adaptability, the team would create abnormal and extreme test cases to ensure that all users are able to successfully interact with and use the product without external help.

1.3. Correction

As the development phase nears completion and relevant tests are conducted to ensure functionality and usability, the team will be correcting bugs and errors that would (inevitably) occur. Below are the maintenance methods that we planned to implement in the development phase.

1.3.1. Preventive Maintenance

Preventive measures such as routine testing and code modularisation to ensure convenient and efficient maintenance of the product.

1.3.2. Corrective Maintenance

Corrective measures, such as debugging and/or error resolving conducted during/after software testing.

1.4.Enhancement

As the product goes under testing, the team would observe potential room for improvements and enhance/upgrade it accordingly. Below are a few aspects that describe the enhancement procedure:

1.4.1. Adaptive Maintenance

The modification of the product to ensure that it remains relevant amongst evolving operational conditions, such as operating systems, hardware, etc.

1.4.2. Perfective Maintenance

Proactive enhancement and modification procedures to ensure that the product meets evolving user needs, and to refine user experience. This is a process done proactively, which means that it revolves around the improvement of software efficiency and convenience, contrary to resolving bugs or errors.

1.5.Best Practices

To ensure that the product is functional, efficient and convenient in all aspects for both the development and deployment phases, the team has agreed upon development practices to adhere to over the duration of the project:

- **Readable Code:** Following naming conventions and maintaining consistency in formatting to reinforce the maintainability of the product.
- **Version Control:** Keeping a rollback option as insurance to ensure software reliability, mainly for after deployment.
- **Modularization:** Maintaining modularized code to enhance efficiency and reliability during the development phase. This would also refine the maintenance and enhancement procedures after deployment.
- **Documentation:** Standardized software documentation is to be preserved to facilitate communication between team members during the development phase and to reinforce reliability during maintenance and enhancement.

2. System Architecture

As mentioned in the previous section, ReportNow was designed under the well-known Model-View-Controller architectural pattern.

2.1. Model Layer

This is where the data structure is defined. The program's databases and their records are included in this layer.

2.2. View Layer

The view layer is where the modelled data is presented to the users, mainly in the form of User Interfaces. It utilises the data available to produce a presentational interface to its users.

2.3. Controller Layer

This layer handles user requests. Through communication between the view and model layers, the controller layer processes the users' input and renders appropriate responses for the view layer to present back to the user.